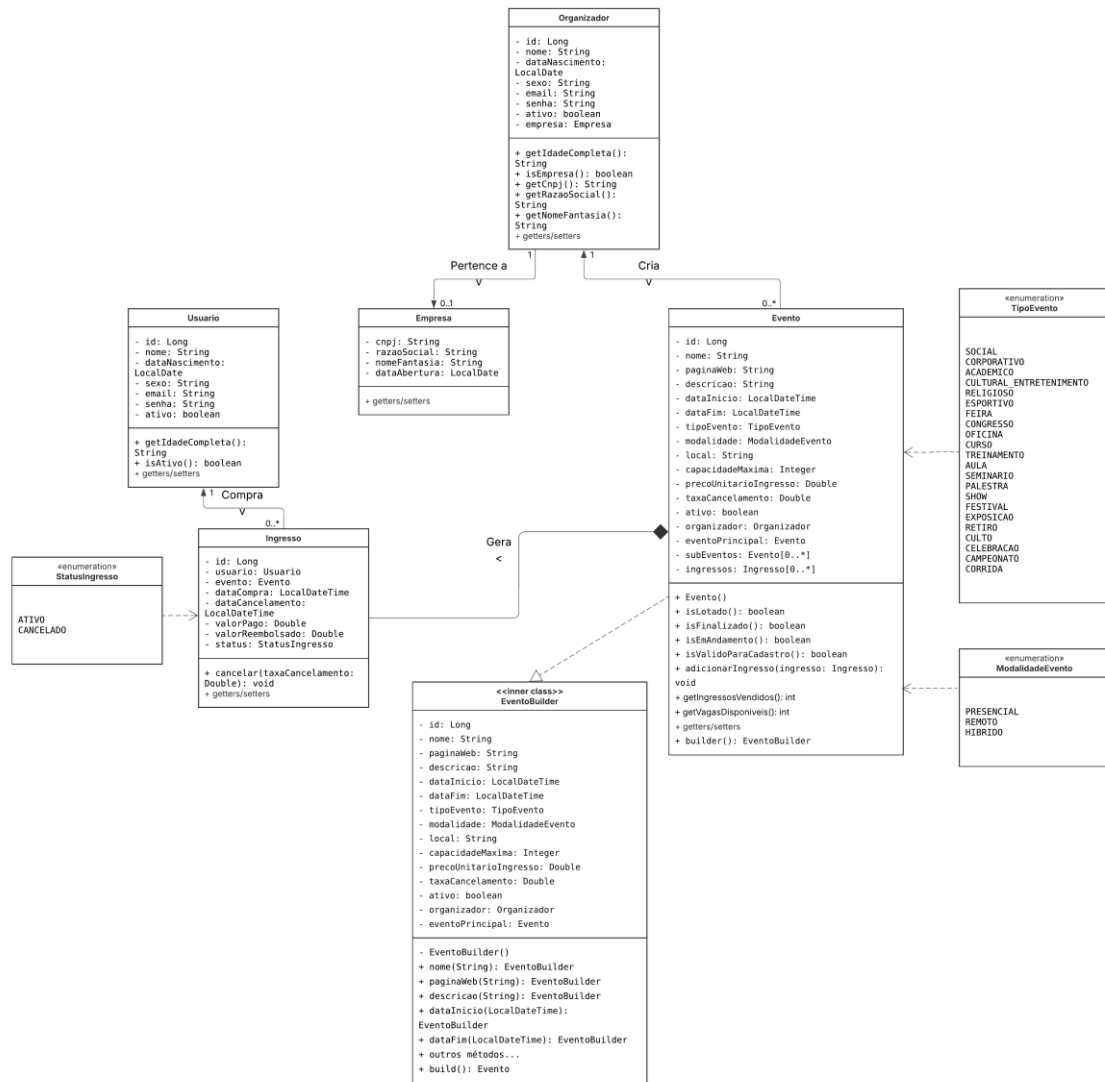




<<inner class>> EventoBuilder
- id: Long - nome: String - paginaWeb: String - descricao: String - dataInicio: LocalDateTime - dataFim: LocalDateTime - tipoEvento: TipoEvento - modalidade: ModalidadeEvento - local: String - capacidadeMaxima: Integer - precoUnitarioIngresso: Double - taxaCancelamento: Double - ativo: boolean - organizador: Organizador - eventoPrincipal: Evento
- EventoBuilder() + nome(String): EventoBuilder + paginaWeb(String): EventoBuilder + descricao(String): EventoBuilder + dataInicio(LocalDateTime): EventoBuilder + dataFim(LocalDateTime): EventoBuilder + outros métodos... + build(): Evento

Controllers

«Controller» UsuarioController
- usuarioService: UsuarioService @RequestMapping: /usuarios
+ cadastrarUsuario(dto): ResponseEntity + alterarUsuario(id, dto): ResponseEntity + visualizarPerfil(id): ResponseEntity + alterarStatus(id, status): ResponseEntity
POST /usuarios PUT /usuarios/{usuarioId} GET /usuarios/{usuarioId} PATCH /usuarios/{usuarioId}/{status}

«Controller» OrganizadorController
- organizadorService: OrganizadorService @RequestMapping: /organizadores
+ cadastrarOrganizador(dto): ResponseEntity + alterarOrganizador(id, dto): ResponseEntity + visualizarPerfil(id): ResponseEntity + alterarStatus(id, status): ResponseEntity
POST /organizadores PUT /organizadores/{organizadorId} GET /organizadores/{organizadorId} PATCH /organizadores/{organizadorId}/{status}

«Controller» EventoController
- eventoService: EventoService @RequestMapping: /organizadores
+ cadastrarEvento(orgId, dto): ResponseEntity + alterarEvento(orgId, eventId, dto): ResponseEntity + alterarStatusEvento(orgId, eventId, st): ResponseEntity + listarEventosDoOrganizador(id): ResponseEntity
POST /{orgId}/eventos PUT /{orgId}/eventos/{eventId} PATCH /{orgId}/eventos/{eventId}/{status} GET /{orgId}/eventos

«Controller» IngressoController
- ingressoService: IngressoService
+ comprarIngresso(orgId, eventId, dto): ResponseEntity + cancelarIngresso(usrId, ingId): ResponseEntity + listarIngressosDoUsuario(id): ResponseEntity
POST /organizadores/{orgId}/eventos/{eventId}/ingressos POST /usuarios/{usrId}/ingressos/{ingId} GET /usuarios/{usrId}/ingressos

«Controller» FeedEventosController
- eventoService: EventoService @RequestMapping: /eventos
+ feedEventos(): ResponseEntity
GET /eventos

DTOs — Request & Response

Request DTOs

«DTO Request»

UsuarioRequestDTO

- nome: String
- dataNascimento: LocalDate
- sexo: String
- email: String
- senha: String

+ getters / setters

«DTO Request»

OrganizadorRequestDTO

- nome: String
- dataNascimento: LocalDate
- sexo: String
- email: String
- senha: String
- cnpj: String
- razaoSocial: String
- nomeFantasia: String

+ getters / setters

«DTO Request»

EventoRequestDTO

- nome: String
- paginaWeb: String
- descricao: String
- dataInicio: LocalDateTime
- dataFim: LocalDateTime
- tipoEvento: TipoEvento
- modalidade: ModalidadeEvento
- local: String
- capacidadeMaxima: Integer
- precoUnitarioIngresso: Double
- taxaCancelamento: Double
- eventoPrincipalId: Long

+ getters / setters

«DTO Request»

IngressoRequestDTO

- usuario: String (email)
- totalPago: Double

+ getters / setters

Response DTOs

«DTO Response»

UsuarioResponseDTO

- id: Long
- nome: String
- dataNascimento: String
- idadeCompleta: String
- sexo: String
- email: String
- ativo: boolean

+ getters / setters

«DTO Response»

OrganizadorResponseDTO

- id: Long
- nome: String
- dataNascimento: String
- idadeCompleta: String
- sexo: String
- email: String
- ativo: boolean
- empresa: EmpresaResponseDTO

«inner class» EmpresaResponseDTO

- cnpj / razaoSocial / nomeFantasia: String

+ getters / setters

«DTO Response»

IngressoResponseDTO

- id: Long
- evento: String
- dataEvento: String
- dataCompra: String
- dataCancelamento: String
- valorPago: Double
- valorReembolsado: Double
- status: StatusIngresso
- eventoAtivo: boolean
- eventoFinalizado: boolean

+ getters / setters

«DTO Response»

EventoResponseDTO

- id, nome, paginaWeb, descricao, dataInicio...
- capacidade, preço, ativo, ingressos, organizador

+ getters / setters

Mappers

«Mapper» UsuarioMapper

— construtor privado —

+ toModel(dto: UsuarioRequestDTO): Usuario
+ updateModel(dto, usuario): void
+ toResponse(usuario: Usuario): UsuarioResponseDTO

Converte: UsuarioRequestDTO ↔ Usuario ↔ UsuarioResponseDTO
Mapeia: nome, dataNascimento, sexo, email, senha, ativo
Calcula: idadeCompleta via Period

«Mapper» OrganizadorMapper

— construtor privado —

+ toModel(dto: OrganizadorRequestDTO): Organizador
+ updateModel(dto, organizador): void
+ toResponse(org: Organizador): OrganizadorResponseDTO

Converte: OrganizadorRequestDTO ↔ Organizador
Cria Empresa se cnpj não for nulo
Mapeia EmpresaResponseDTO (inner class)
Email não pode ser alterado (updateModel)

«Mapper» EventoMapper

— construtor privado —

+ toModel(dto: EventoRequestDTO): Evento
+ updateModel(dto, evento): void
+ toResponse(evento: Evento): EventoResponseDTO

Converte: EventoRequestDTO ↔ Evento ↔ EventoResponseDTO
Mapeia: nome, datas, tipo, modalidade, local, preço
Inclui: organizador(nome), eventoPrincipal(id+nome)
Calcula: ingressosVendidos, vagasDisponiveis
Formata: LocalDateTime → String (toString)

«Mapper» IngressoMapper

— construtor privado —

+ toResponse(ingresso: Ingresso): IngressoResponseDTO

Converte: Ingresso → IngressoResponseDTO
Mapeia: id, valorPago, valorReembolsado, status
Formata: dataCompra, dataCancelamento → String
Acessa Evento: nome, dataInicio, ativo, finalizado
Não possui toModel (criação via construtor direto)
Não possui updateModel